

Relación 4

Ejercicio 1. Acceso a métodos en Ruby

Sea el siguiente código en Ruby:

```
class Ejemplo
  def publico_implicito_protegido
    metodo_protegido
  end
  def publico_explicito_protegido
    self.metodo_protegido
  end
  def publico_implicito_privado
    metodo_privado
  end
  def publico_explicito_privado
    self.metodo_privado
  end

  protected
  def metodo_protegido
    puts "metodo protegido"
  end

  private
  def metodo_privado
    puts "método privado"
  end
end
```

1. Indica cuál es el resultado de ejecutar las siguientes líneas de código

a) Ejemplo.new.publico_implicito_protegido	Sin problema pq público
b) Ejemplo.new.publico_explicito_protegido	''
c) Ejemplo.new.metodo_protegido	Error. desde fuera de una clase no puedo llamar a protegido
d) Ejemplo.new.publico_implicito_privado	Sin prob. pq público
e) Ejemplo.new.publico_explicito_privado	''
f) Ejemplo.new.metodo_privado	error

2. Explica el resultado obtenido en cada uno de estos puntos

dentro del mismo paquete es público.

Ejercicio 2. Visibilidad de métodos en Java

Sea el siguiente código en Java:

```
public class Ejercicio13 {  
  
    public void publico(){  
        System.out.println("método público");  
    }  
    void paquete(){  
        System.out.println("método paquete");  
    }  
    protected void protegido(){  
        System.out.println("método protegido");  
    }  
    private void privado(){  
        System.out.println("método privado");  
    }  
}
```

Prueba el siguiente código en la misma clase, en otra clase del mismo paquete y en otra clase de distinto paquete:

```
public static void main(String[] args) {  
    Ejercicio13 e13 = new Ejercicio13();  
    e13.publico();  
    e13.protegido();  
    e13.paquete();  
    e13.privado();  
}
```

En este caso
todo se imprime
bien pq es desde
la misma clase.

Plantéate qué ocurre en cada una de las situaciones indicadas.

- Desde otra clase del mismo paquete: todo bien menos privado y protegido
- Desde otra clase de distinto paquete: Solo el público bien.
- Desde otra clase que hereda de esta, pero en otro paquete: Protegido y público bien.

Ejercicio 3. Visibilidad de variables en Java

Sea el siguiente código en Java:

```
public class Ejercicio14 {  
    private int i=0;  
    int j= 1;  
    protected int k=2;  
    public int l=3;  
}
```

Prueba el siguiente código en la misma clase, en otra clase del mismo paquete y en otra clase de distinto paquete:

```
public static void main(String[] args) {  
    Ejercicio14 e14 = new Ejercicio14();  
    1 System.out.println("privada : " + e14.i);  
    2 System.out.println("paquete : " + e14.j);  
    3 System.out.println("protected : " + e14.k);  
    4 System.out.println("public : " + e14.l);  
}
```

Plantéate qué ocurre en cada una de las situaciones indicadas.

misma clase : todos correcto

Otra clase igual paquete : 1. error. necesita consultar

2. ✓

3. ✓

4. ✓

Distinto paquete: 1. Error

2. Error

3. Error

4. ✓

Dentro de un mismo
paquete se puede
acceder a #

Ejercicio 4. Dada la siguiente clase en Java:

```
public class Vertebrado
{...
public ArrayList<String> partesDelAbdomen();
public void desplazarse();
public String comunicarse(Vertebrado vertebrado);
}
```

Indica con una cruz en la casilla correspondiente según si la declaración de los siguientes métodos de la clase Mamifero, subclase de Vertebrado, sobrecargan (overloading) o redefinen (overriding) a los de la clase Vertebrado:

	Sobrecarga / Overloading	Redefinición / Overriding
public ArrayList<String> partesDelAbdomen();		×
public void desplazarse(Modo m);	×	
public String comunicarse(Vertebrado vertebrado);		×
public String comunicarse(Mamifero mamifero);	×	

Ejercicio 5. A partir de las siguientes clases, implementa la clase *Alumno-Informatica* que hereda de *Alumno*. Debe tener una nueva variable de instancia que es una colección de strings con los *dispositivos* que utiliza: como por ejemplo {PC, tablet, smartphone}. Esa variable debe poder consultarse y modificarse. También debe heredar el método *estudiar()*, pero modificando su implementación para que además de estudiar como los otros alumnos, diga que estudia con el último dispositivo de su colección. No olvides implementar su constructor. Implementa este mismo problema en Ruby.

```
public class Persona {
    protected String nombre ;
    public Persona(String nom) { this.setNombre(nom); }
    protected String getNombre() { return this.nombre; }
    protected void setNombre(String nom) { this.nombre=nom; }
    public String hablar(){ return 'bla bla bla';}
}

public class Alumno extends Persona {
    public String carrera;
    public int curso;

    public Alumno(String nom, String carr,int cur)
    {
        super(nom);
        carrera=carr;
        curso=cur;}
    public void estudiar() { System.out.println('estudiando'); }
}
```

Java

```
public class alumno-informatica extends Alumno {

    private ArrayList<String> d;

    Alumno-Informatica(String nom, String carr, int cur){
        super ( nom, carr, cur);
        d = new ArrayList<>();
    }

    public ArrayList<String> getD () {
        return d;
    }

    public void setD (String nuevo) {
        d.add (nuevo);
    }

    public void estudiar {
        super();
        System.out.print ("On " + d.get (d.size()-1));
    }
}
```

Ruby

```
public class Alumno-Infomatica
```

```
  def initialize ( nom, carr, curs)
```

```
    super ( nom, carr, curs)
```

```
    @d = []
```

```
  end
```

```
  def get_D
```

```
    @d
```

```
  end
```

```
  def set_D ( nuevo)
```

```
    d.add (nuevo)
```

```
  end
```

```
  def estudiar
```

```
    super
```

```
    puts "Con #{@d.size-1}"
```

```
  end
```

Ejercicio 6. Dada la siguiente clase abstracta en Java,

```
abstract class Transporte { → para constructor.  
    protected String marca;  
    protected String getMarca() { return this.marca; }  
    protected void setMarca(String marc) { this.marca=marc; }  
    public abstract String hacerRuta(String origen, String destino);  
}
```

Crea dos nuevas clases, por ejemplo Bicicleta y NaveEspacial, que hereden de ella y que implementen el método *hacerRuta* de diferente forma cada una. Dibuja el diagrama de clases UML correspondiente a este ejercicio.

```
public class Bicicleta extends Transporte {
```

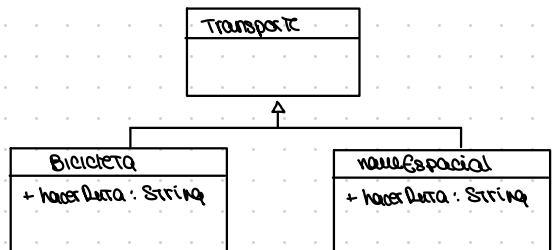
```
    @Override
```

```
    public String hacerRuta (String origen, String destino) {
```

```
        return "Ruta en bici desde " + origen + " hasta " + destino;
```

```
    }
```

NaveEspacial queda igual.



Ejercicio 7. Haz el ejercicio anterior en Ruby.

```
class Bicicleta < Transport
  def hacerRuta(Origen, destino)
    puts "Viaja en bici desde #{Origen} hasta #{destino}"
  end
end
```

Ejercicio 8. Implementa en Java y en Ruby una jerarquía de herencia en la que estén representados los distintos tipos de instrumentos musicales: viento, percusión y cuerda. Además, los de viento pueden ser de madera o de metal. Todas las clases deben tener implementados los siguientes métodos:

tocar(String nota): muestra la nota que se le pasa (do, re, mi, fa, so, la si).

ajustar(): muestra "soy el instrumento fulanito y me estoy ajustando".

queSoy(): muestra el tipo de instrumento que es.

Escribe un pequeño main() en el que podamos oír a la orquesta.

Java:

```
public abstract class Instrumento {
    public String tocar(String nota) {
        return nota;
    }
}
```

```
public abstract void ajustar();
public abstract void queSoy();
}
```

```
public class Percusión extends Instrumento {
    @Override
    public void ajustar() {
        System.out.print("Soy Percusión y me estoy ajustando");
    }
}
```

```
@Override
public void queSoy() {
    S.o.p("Soy Percusión");
}
}
```

```
public class Viento extends Instrumento {
    @Override
    public void ajustar() {
        System.out.print("Soy Viento y me estoy ajustando");
    }
}
```

} puede ser así? en lugar de ser como el resto?
Pueden construirse!

falta ruby.

```
@Override
Public void queSoy() {
    S.o.p("Soy Uienro");
}
```

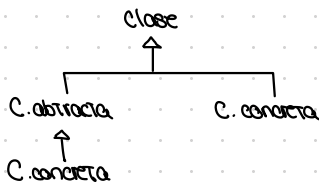
```
Public class Cuerda extends Instrumento {
    @Override
    Public void ajustas() {
        System.out.print("Soy Cuerda y me estoy ajustando");
    }
}
```

```
@Override
Public void queSoy() {
    S.o.p("Soy Cuerda");
}
```

```
Public class Uienromadera extends Uienro {
    Public void queSoy() {
        super.queSoy();
        S.o.p("y soy de madera");
    }
}
```

```
Public class Uienrometal extends Uienro {
    Public void queSoy() {
        super.queSoy();
        S.o.p("y soy de metal");
    }
}
```

Ejercicio 9. Representa mediante un diagrama de clases las relaciones entre los distintos tipos de clases que podemos encontrarnos en Java: Clase, ClaseAbstracta, ClaseConcreta, ClaseHoja y ClaseNoHoja, siendo ClaseHoja aquella que está al final de una jerarquía de herencia (aquella que ya no tiene subclases).



C hoja solo puede ser concreta, una no hoja puede ser abstracta.

Abstracta: no se puede instanciar y es la base para otras.

Concreta: Puede derivar de una clase normal o abstracta.

Clase hoja: No tiene subclases.

Ejercicio 12. Teniendo en cuenta el siguiente diagrama de clases, señala en la segunda columna de la tabla aquellas líneas de código que contienen un error de compilación por incompatibilidad de tipos (ECIT) y escribe en la tercera columna la corrección necesaria para evitarlos.



Código	ECIT	Corrección
Girable arti=new Artista();	X	Girable arti = ((cantante) new Artista());
Cantante cant1=new Cantante();	✓	
Cantante sol=new Solista();	✓	
Solista cant2=new Cantante();	X	Solista cant2 = new (solista)cantante();
Bailarin bail= new Artista();	X	Bailarin bail = (Bailarin) new Artista();

→ Eso pone Bca.

Ejercicio 13. Teniendo en cuenta:

- el diagrama de clases del ejercicio anterior,
- que se han resuelto todos los errores del ejercicio,
- que en Java la primera posición de los contenedores es la 0,
- que todos los artistas actúan, pero sólo los cantantes cantan y sólo los solistas cantan solos,

marca las líneas donde se produce un error, indicando en la segunda columna si es de compilación (C) o de ejecución (E) y en la tercera el código correcto.

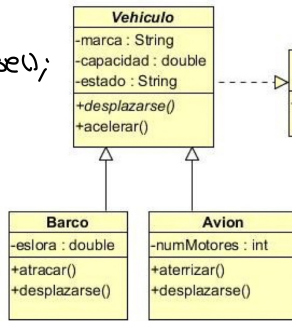
Código	Error(C/E)	Corrección
List<Artista> lista=new ArrayList();	✓	
lista.add(arti);		→ lista.add(((cantante)artista)arti)
lista.add(cant1);	C	
lista.add(sol);		
lista.add(cant2);		
lista.add(bail);		
lista.get(1).canta();		((cantante)lista.get(1).canta());
lista.get(0).actua();	✓	
(Solista) lista.get(3).cantaSolo();	✓	

la lista siempre devuelve artistas.

Ejercicio 15. Dado el siguiente diagrama de UML, impleméntalo en Java. Presta atención a que el nombre de la clase Vehículo y su método desplazarse() aparecen en cursiva.

Interface Transporte

public void desplazarse();



Public abstract class Vehiculo implements Transporte

private String marca;
private double cap;
private String estado;
public abstract void desplazarse();
public void acelerar();

public class Barco extends Vehiculo
private double eslora;
public void atracar();
@Override
public void desplazarse();

Ejercicio 16. Teniendo presente el diagrama anterior, indica cuáles de los siguientes trozos de código son correctos y cuáles darían error de compilación o de ejecución. En caso de que den error, indica por qué y cómo lo solucionarías.

Código	Error y si procede corrección
Transporte x = new Barco(); x.atracar();	((Barco)x).atracar(); atracar no está en vehículo
Avion av = new Avion(); av.acelerar();	hereda acelerar de vehículo.
Vehiculo v = new Vehiculo(); v.desplazarse();	No puedo instanciar una clase abstracta.
Vehiculo v2 = new Vehiculo(); v2.acelerar();	"
Transporte t = new Avion(); Barco b= new Barco(); t = b;	No da error pq transporte está por encima de Barco.
Avion a = new Avion(); String est = a.estado;	estado es privado Error.
Vehiculo v3 = new Barco(); ((Barco) v3).atracar();	Aunque v3 se inicialice como Barco sigue siendo un vehículo así que si es necesario el cast.
List<Transporte> listaTransportes = new ArrayList(); listaTransportes.add(new Barco()); listaTransportes.add(new Avion()); listaTransportes.add(new Barco()); for(Transporte tr: listaTransportes){ tr.acelerar(); tr.atracar();}	Dará error porque transporte conoce acelerar pero no atracar(aunque sea instancia de Barco) no podemos hacer un cast que solucione el error pq cuando solucionamos para como falla para avión.
List<Transporte> otraLista = new ArrayList(); otraLista.add(new Barco()); otraLista.add(new Avion()); otraLista.add(new Barco()); for(Transporte tr: otraLista){ ((Barco) tr).acelerar(); ((Barco) tr).atracar();}	Error ejecución pq no todos los tipos dinámicos son Barcos.

Ejercicio 17. A partir del siguiente diagrama de clases y del código proporcionado, detecta errores de compilación y de ejecución. Corrígelos para que funcione correctamente.

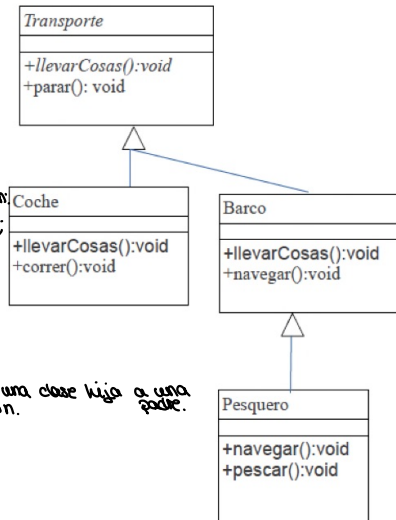
```
class PruebaLigadura {
    public static void main ( String args[]){

        Coche c1 = new Coche();
        c1. llevarCosas();
        c1.correr();

        Barco b = new Barco();
        b.llevarCosas();
        b.navegar();
        b.correr();
        Pesquero p = new Pesquero();
        p.navegar();
        p.pescar();
        p.llevarCosas();
        p=b;
        b=p;
        b.navegar();
        b.pescar();

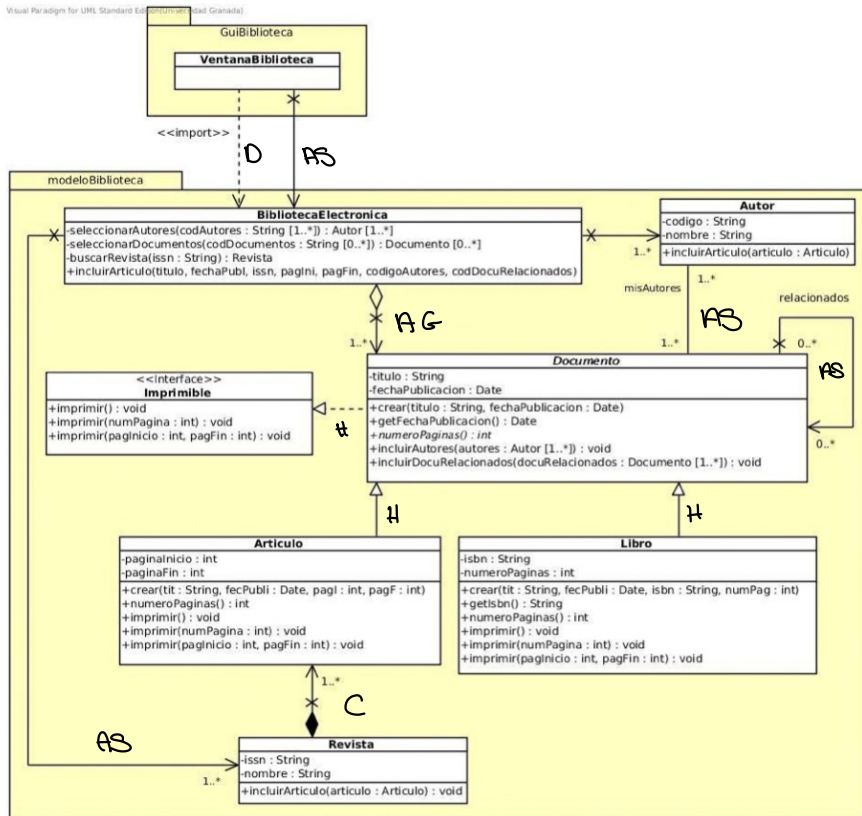
        ArrayList v= new ArrayList();
        v.add(c1);
        v.add(p);
        (v.get(1)).navegar();
        (v.get(1)).llevarCosas();

    }
}
```



error compilación: (barco)c1; → **solución al error de compilación:** `((coche)(Transporte)b).correr();`
para si que dando error de ejecución.
Error compilación: no se puede igualar una clase hija a una padre.
solución `p = ((Pesquero)b → err. ejecución.`
`(v.get(1)).navegar();` → `((Pesquero)v.get(1)).navegar();`

Ejercicio 18. A partir de los siguientes diagramas de clases, resuelve las cuestiones que se plantea (siempre que sean de codificación hazlo en Java y en Ruby):



- Define la clase *Documento* (cabecera, atributos y cabeceras de los métodos).

lo vamos crear?

Public abstract class Documento implements Imprimible {

public String titulo;
public Date fechaPublicacion;
public ArrayList<Autor> misAutores;
public ArrayList<Documento> relacionadas;

public Date getFechaPublicacion() {

public abstract int numeroPaginas();

public void incluirAutores(ArrayList<Autor> autores) {

public Documento(String titulo, Date fp) {
this.titulo = titulo;
fechaPubl = fp
misAutores = new ArrayList<>();
relacionados = new ArrayList<>();

}

@Override

public void imprimir() {} // igual para todas las imprimir.

no existe, para ruby, interfaz ni clase abstracta. las implemento como clases normales.

```
class Documento < imprimible
```

```
  def initialize(t, fp)
```

```
    @titulo = t
```

```
    @fechaPubli = fp
```

```
    @misAtributos = []
```

```
    @relacionados = []
```

```
  end
```

```
  def get_fecha
```

```
  end
```

num_paginas como es abstracto no lo implemento aqui, solo en los hijos.

imprimir tampoco porque lo hereda de Imprimible, aqui estarian implementados pq lo cuantos como si fuera una clase padre normal (no interfaz)

```
  def incluir autores (autores)
```

```
  end
```

```
end
```

- Define la clase *Articulo* (cabecera, atributos y cabeceras de los métodos).

```
public class Articulo extends Documento
```

```
  public int paginaInicio;
```

```
  public int paginaFin;
```

```
  public void Articulo(String tit, Date fechaPubli, int pagI, int pagF) {
```

```
    super(tit, fechaPubli);
```

```
    this.paginaInicio = pagI;
```

```
    this.paginaFin = pagF;
```

```
}
```

```
@Override
```

```
public int numPaginas() { // igual para el resto.
```

```
}
```

```
class Articulo < Documento
```

```
  def initialize (tit, fechaPubli, pagI, pagF)
```

```
    super (tit, fechaPubli)
```

```
    @paginaInicio = pagI
```

```
    @paginaFin = pagF
```

```
  end
```

```
  def numPaginas
```

```
  end
```

```
  def imprimir
```

```
  end
```

```
  def imprimir (num-pagina)
```

```
  end
```

```
end
```

- Implementa el constructor que se indica en la clase *Articulo*.

Hecho en el apartado anterior.

- Define la interfaz *Imprimible*.

```
public interface Imprimible {
    public void imprimir();
    public void imprimir ( int numPagina );
    public void imprimir ( int paginico , int pagfin );
}
```

En ruby no hay interfaces

- Indica los atributos que definen el estado de la clase *BibliotecaElectronica*.

```
Arraylist < Revista > revistas;
Arraylist < Documento > documentos;
```

```
Arraylist < Autor > autores;
```

Revisar con re12.

- La clase *Documento* figura en cursiva, lo que indica que es abstracta. Indica los dos motivos por los que lo es.

- En este modelo, ¿puede haber artículos que no estén ligados a una revista?

No, porque al ser composición no pueden existir artículos sin revista.

- Escribe el contenido del fichero *VentanaBiblioteca* completo (pero sin añadir nada que no aparezca en el diagrama).

```
package GuiBiblioteca {
    import modeloBiblioteca.BibliotecaElectronica;
    public class VentanaBiblioteca {
        public BibliotecaElectronica be;
    }
}
```

```
require_relative 'bibliotecaElectronica.rb'
module GuiBiblioteca
    class VentanaBiblioteca
        end
    end
end
```

- En la clase *Articulo*, indica cuáles de sus métodos están sobrecargados o redefinidos. Justifica tu respuesta.

Todos redefinidos.

- Corrige el código:

```
Imprimible docu = new Documento("Intemperie", fecha); // suponiendo que
fecha está inicializada
docu.imprimir();
```

No se puede instanciar una clase abstracta

```
Documento docu = new Libro("ISBN10102030");
String codigo = docu.getIsbn();
```

↳ faltarán atributos.

```
((Libro)docu).getIsbn();
```

- Rellena la siguiente tabla indicando el tipo estático y dinámico de la variable *docu* en las siguientes líneas de código:

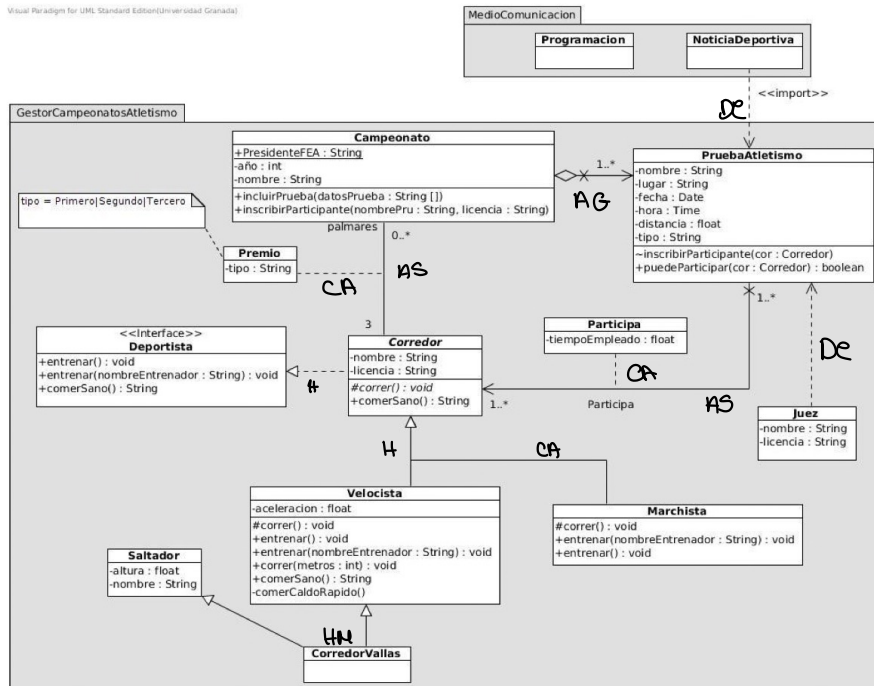
	Tipo estático	Tipo dinámico
1. Documento docu = new Artículo(titulo, fecha, pagIni, pagFin);	Documento	Artículo
2. docu.imprimir(pagIni, pagFin);	"	"
3. docu = new Libro(titulo, fecha, isbn, pags);	"	Libro
4. docu.imprimir();	"	"

- Indica si hay errores de compilación o ejecución en el código anterior (suponiendo que las variables *titulo*, *fecha*, *pagIni*, *pagFin*, *isbn* y *pags* han sido declaradas e inicializadas convenientemente con anterioridad). Justifica tu respuesta.

- Rellena la siguiente tabla marcando con una "X" la situación que corresponda a cada una de las líneas del siguiente bloque de código (suponiendo que las variables *titulo*, *fecha*, *pagIni* y *pagFin* han sido declaradas e inicializadas convenientemente con anterioridad).

	Ningún error	Sólo error de compilación	Sólo error de ejecución
Imprimible imp = new Artículo(titulo, fecha, pagIni, pagFin);	X		
Documento docu = imp; (documento).imp		X	
imp.numeroPaginas(); ((Libro) imp).		X	
((Libro) docu).getIsbn();	X		

Ejercicio 19. Partiendo del siguiente diagrama de clases:



- Responde V (Verdadero) o F (Falso) a las siguientes cuestiones:

Desde la clase NoticiaDeportiva se puede acceder a todos los elementos públicos del paquete GestorCampeonatosAtletismo directamente	✓
Un CorredorVallas es un Velocista y Saltador	✓
El estado de un objeto de la clase Juez no viene determinado por el estado de un objeto de la clase PruebaAtletismo	✓
Una PruebaAtletismo puede existir sin estar asociada a la clase Campeonato	✓
Un Velocista es un Corredor	✓
Un Deportista es un Corredor	F
La clase Corredor tiene 4 métodos, 3 abstractos y 1 concreto	✓
La clase Marchista está mal representada en el diagrama, debe ser abstracta	F
La clase CorredorVallas presenta un conflicto de nombres	✓
Todas las instancias de la clase Campeonato tienen una copia de la variable PresidenteFEA	F

no son publicas

no es composición

concreta todos los métodos abstr. si tuviera eso ya no sería

Es de clase, voy a usar una esa variable.

- Usando la siguiente nomenclatura: AS = Asociación, CO = Composición, AG = Agregación, DE = Dependencia, CA = Clase Asociación, RE = Realización, HE = Generalización o Herencia y HM = Herencia múltiple, etiqueta los elementos correspondientes en el propio diagrama de clases.
- Marca de los siguientes cuáles son métodos abstractos en Marchista:

correr()	
comerSano()	
entrenar()	
entrenar(nombreEntrenador:String)	

- Marca qué métodos están redefinidos y/o sobrecargados en la clase Velocista.

	redefinido	sobrecargado
correr()	✗	
comerSano()	✗	
entrenar()	✗	

- Proporciona el tipo estático y dinámico de la variable que se indica en las siguientes sentencias Java.

	Variable	Tipo Estático	Tipo Dinámico
Deportista c= new Velocista();	c	Deportista	Velocista
Corredor d= new Marchista();	d	Corredor	Marchista
c=new Marchista();	c	Deportista	Marchista
d = c;	d	Corredor	Velocista

La clase: marchista

- En el siguiente código Java:
 - Corrige los posibles errores de compilación.
 - Una vez corregidos los errores de compilación, indica las líneas de código en las que habría error de ejecución.

	Corrección del error en Compilación	Error en ejecución	
Corredor c= new Velocista();			
Deportista d= new Marchista();			
d.comerSano();			usa comerSano de corredor
Marchista m= d;	Marchista m= (Marchista)d	no se ejecutaria.	
d=c;			
d.correr(150);	((Velocista)d).correr(150)		correr es abstracto y no está overload en Marchista.
Velocista v = d;	Velocista v = (Velocista)d	no solución	
ArrayList<Corredor> corredores = new ArrayList();			
corredores.add(c);			
corredores.add(m);			
corredores.get(0).correr(10);	((Velocista)corredores).		
corredores.get(1).correr(10);	((Marchista) corredores).	→ no puede usar correr con parametro.	
c= new Corredor();	no se pueda instanciar clase abstracta.		
compila con error a velocista pero error ejec.			

- Completa el código de la siguiente clase parametrizada en Java, la cual representa a un club de corredores de cualquier tipo: velocistas o marchistas.

$\rightarrow$ T extends corredor

```

public class Club<.....> {
    private Map<String, ..T..> miembros; // key = número de licencia
    private T. lider;

    // Constructor...
    public Club(..T. lider) {
    }
    // Consultor de un miembro del club a partir del numero de licencia
    public ..T. getMiembro(String numerolicencia){
    }
    // Incluir un nuevo miembro en el club
    public void incluirMiembro(..T. miembro){
    }
    // cambiar el lider
    public void setLider(..T. lider) {
    }
    // Obtener el lider
    public ..T. getLider(){
    }
}

```

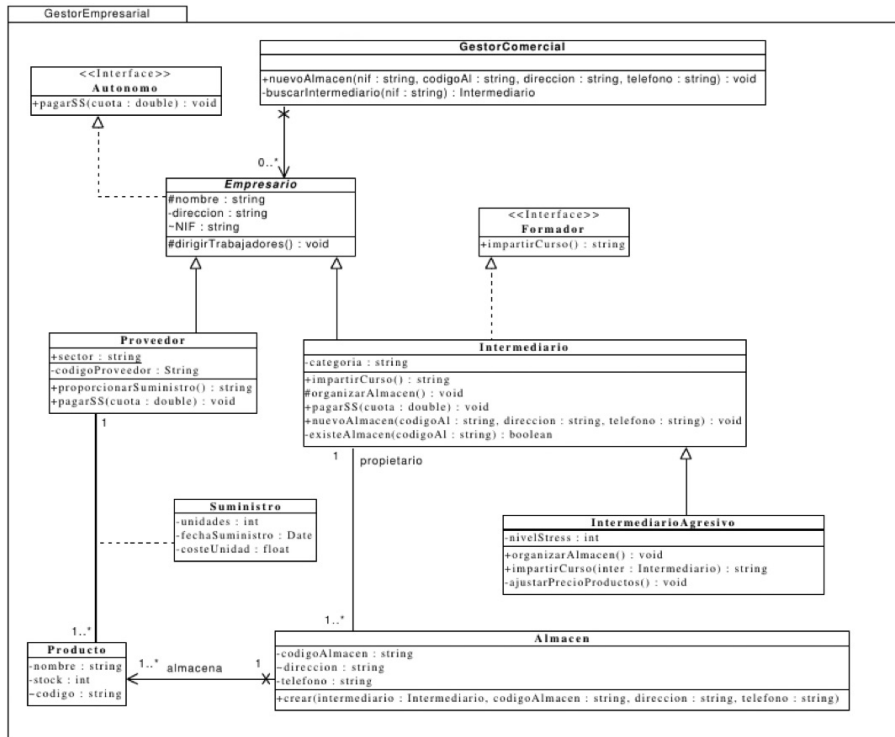
- Escribe un pequeño main en el que se use la clase Club<T> como club de Velocistas

```

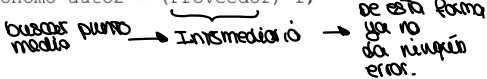
public static void main (String[] args) {
    Velocista v= new Velocista ("Eva", " ", 34);
    Velocista v2= new Velocista ("Maria", " ", 45);
    Club<Velocista> ClubVelocistas = new Club(v);
    ClubVelocistas.incluirMiembro (v2);
    ClubVelocistas.setLider (v2);
}

```

Ejercicio 20. Examen 16/17. Dado el diagrama:



Indica si las siguientes líneas de código Java producen error de compilación, de ejecución, ambos o ninguno. Supón que están en un main en una clase nueva dentro del mismo paquete. Si hay error explica cómo lo arreglarías y si no hay error, indícalo explícitamente

Considera para este ejercicio que todas las clases tienen un constructor válido que no recibe atributos.	Error de compilación	Error de ejecución
Formador f = new Intermediario();		
f.pagarSS(23.4);	((Intermediario)f).pagarSS()	
Autonomo auto1 = f; 	Error porque formador no descender de autonomo. autonomo auto = (Intermediario)f	
Autonomo auto2 = (Proveedor) f; 	Da porque proveedor no está en su línea descendente solución	
IntermediarioAgresivo empl = new IntermediarioAgresivo(); empl.ajustarPrecioProductos();	Error compilación porque método privado.	
Empresario emp2 = new Empresario();	no se puede instanciar abstracta.	
ArrayList<Formador> formadores = new ArrayList(); formadores.add(f); formadores.add(empl); → no da error porque IA es un formador.		
formadores.get(1).impartirCurso();		
formadores.get(1).impartirCurso(f);	E.C. porque en un formador no existe ese método. Igualmente lo estaría mal pq se le pasa un formador →(Intermediario)f);	
((IntermediarioAgresivo) formadores.get(0)).impartirCurso(empl);	no da error porque aunque se le pase un IA, es también un intermediario.	Si

no corregido pero creo que es así.

Ejercicio 10. Implementa en Java una clase paramétrica a partir de la cual se puedan definir clases de grupos de personas con un líder, como por ejemplo grupos de música con un cantante, o empresas con un jefe. Implementa también alguna de esas clases a partir de la clase paramétrica definida.

```
public class grupo<T>{  
    private T lider;  
    private ArrayList<T> integrantes;  
  
    public grupo(T lider){  
        this.lider = lider;  
        integrantes = new ArrayList<T>();  
        integrantes.add(lider);  
    }  
  
    public void addIntegrante (T nuevo){  
        integrantes.add(nuevo);  
    }  
  
    public void cambiaLider (T nuevoLider){  
        this.lider = nuevoLider;  
        addIntegrante(nuevoLider);  
    }  
}
```

....En algun lugar está implementada la clase empresario....

```
public class Empresa extends grupo<Empresario>{  
  
    public void Empresa (Empresario lider){  
        super(lider);  
    }  
  
    (... resto de métodos de lo que haga Empresa)  
}
```

Ejercicio 11. Partiendo del siguiente diagrama de clases, modifícalo añadiendo una interfaz *Prestable* implementada por la clase *ElementoPrestable* y sus subclases. Añade a este diagrama de clases los atributos y operaciones de las distintas clases y de la interfaz *Prestable*.

